Uniform Zero and Constructed Premises for Intuitionistic Primitive Recursive Arithmetic

Complete rules, interpretation proofs, and a reproducible comparison with the current drafts

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

We give a uniform numerical representation in which every canonical term, including zero, has a true Python Boolean value. A small change to the existing opening operation makes all generated zero instances use the same representation. Consequently zero initializers, zero successor predecessors, and zero equality sources need no auxiliary proof. Equality and conditional universal majors can also be constructed from their instruction fields, removing repeated formula data from certificates. The construction preserves the complete native consequence relation and therefore the inherited whole-domain bi-interpretation with full first-order iPRA.

The inspected repository snapshot contains 17 current archives. Its smallest delivered complete checker occupies 558 normalized Python bytes. A composition of existing implementation techniques reduces that program to 545 bytes; uniform zero costs five additional bytes, and constructed majors cost six more. Thus the new complete profiles occupy 550 and 556 bytes. A matched comparison is essential: neither new representation is smaller than the 545-byte implementation of the old representation. The numerical and logical axioms are inherited. A significant new axiomatic foundation, an absolute minimum, or a single program improving all existing size measures has not been established.

The positive result is a proof-preserving certificate normalization, with exact savings for the displayed compiler and reproducible checking evidence. The archive includes the complete primitive inventory, all inherited arithmetic and logical bridge proofs with attribution, standalone checkers, proof producers, examples, negative controls, source measurements, and maximum-compression gzip measurements as information only.

Research status. The requested foundational breakthrough remains open. This report supplies a complete equivalent presentation and a smaller implementation family, while distinguishing inherited implementation savings from the new certificate theorem. No proof-assistant verification or absolute simplicity claim is made.

Contents

1 The comparison target and the result **3**

2 Complete mathematical presentation **4**

3 Why the presentation is fully bi-interpretable with iPRA **6**

4 A uniform zero representation and constructed majors **8**

5 The complete checker and its trust boundary **13**

6 Reproduction and validation **15**

7 Further research questions **16**

Attribution and the full inherited bridge **17**

A Complementary introductions with one local assumption **17**

B Arithmetic strength and both ordinary interpretation composites **22**

C Internal scalar predecessor bootstrap **24**

D From literal-equation induction to full intuitionistic PRA **28**

E Pairing, primitive recursion, and internal numerical composites **34**

1 The comparison target and the result

All repository comparisons are pinned to ProveIt commit

`76a2651601d17b34d46753f189ea026a21ce8d29`.

Every direct archive in `docs/drafts` was downloaded and checked against its Git blob SHA and byte length. A recursive census found 17 direct archives, 32 distinct ZIP payloads, 2,239 Python-source occurrences after archive-content deduplication, and 443 distinct Python byte strings. The complete manifest and source census are in `evidence/`. The comparison includes complete reference-directory checkers, not just the principal file advertised by each draft.[1, 2]

The supplied corpus has separate attained minima of 558 normalized source bytes, 262 significant tokens, and 296 abstract-syntax-tree nodes. These are attained by different programs. They are upper bounds on an unknown optimum, not a mathematical lower-bound theorem. The token/AST leader uses generic exception-based traversal and depends on the actual Python recursion limit to reject some nonempty strings. The exact-list programs used here do not need that particular termination qualification.

Complete program	Bytes	Tokens	AST	gzip-9
Published byte leader	558	264	310	345
Published explicit-list	566	264	308	351
Published token/AST leader	598	262	296	350
Matched inherited control	545	267	311	336
Control, explicit-list	553	267	309	343
Uniform zero	550	269	314	338
Uniform zero, explicit-list	558	269	312	344
Constructed majors	556	272	314	341
Constructed, explicit-list	564	272	312	346
Readable constructed checker	1600	309	361	891

Bytes, tokens, and AST refer to the normalized complete module. The gzip column counts the delivered raw source. The readable source is 2,192 raw bytes; normalized gzip sizes are also recorded in `evidence/metrics.json`. The published generic token/AST profile has the malformed-string qualification described in the text.

The controlled comparison has three stages. First combine previously documented lexical scopes, an immutable default context, the invalid `0j` missing-image sentinel, and a shared continuation written with Boolean selection. The result implements the existing certificate representation and measures 545 bytes. Second normalize numerical zero uniformly, at a cost of five bytes. Third construct equality and closure majors rather than repeating them, at a further cost of six bytes. The resulting 550- and 556-byte programs are smaller than the literal 558-byte supplied file, but larger than the matched 545-byte control. That distinction is part of the result, not an omitted caveat.

The explicit-list versions use `isinstance(a,list)`. The smaller versions use the already published predicate `a*0==[]` on values returned by Python's JSON decoder. Both are standalone programs: no producer, comparison tool, mathematical oracle, or trusted axiom file is imported. Python and its standard JSON library are the common platform excluded by the corpus's convention.

1.1 What is new relative to the inspected collection

The current collection already knows nonempty terms provably equal to zero, truthy scalar aliases for zero, unary paired formulas, triplicated closures, the affine arity traversal, scalar iteration, and complementary case/progress introduction. We do not claim any of these separately as new.

The new combination is to choose a truthy zero *uniformly throughout the canonical language and the checker's generated data*, prove that opening commutes with that choice, and exploit it in premise construction. Merely submitting a different zero alias to the old checker would not establish this theorem: its generated base assumptions would still contain its old zero spelling. The scalar normalizer resolves precisely that mismatch.

The numerical language, arithmetic equations, native logical meaning, equation-only induction restriction, and interpretation domain do not change. The contribution is therefore a new certificate presentation of the inherited theory, with a quantitative proof-normalization result. It is not a new foundation for arithmetic. Novelty is asserted only relative to the inspected current and embedded draft texts, not to all published literature.

2 Complete mathematical presentation

We use full first-order intuitionistic PRA: defining equations for all fixed primitive-recursive descriptions, the successor axioms, intuitionistic first-order equality logic, and contextual quantifier-free induction. This target is distinct from merely its equational or negative fragment. The standard distinction between first-order intuitionistic, first-order classical, and quantifier-free presentations is explicit in the background literature.[12]

2.1 One numerical sort and two arithmetic schemas

Numerical terms have the grammar

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Write $1 := S0$. Only the update body h binds u. Its other parameters are fixed through iteration. The counter is not an extra argument supplied to the update. Equivalently, for each fixed earlier-stage term $h(u, \vec{p})$, introduce a first-order symbol $i_h(n, a, \vec{p})$ at the next finite syntactic stage. There is no quantified numerical code for an arbitrary update and no uniformly total universal primitive-recursive evaluator.

The only primitive nonlogical schemas are

$$I(0, a; u.h) = a, \tag{I0}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

The substitutions are capture-free. Either schema may be used in any retained context. Both remain explicitly present in the new presentation.

2.2 Native formulas and intended meanings

Formulas are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A,$$

with distinct bound variables in a paired formula. Contexts are finite, reusable lists of formulas. The designated contradiction and truth are $\bot_* = (0 = S0)$ and $\top_* = (0 = 0)$. The ordinary readings are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x (B^\circ(x,0) \to \forall y\, B^\circ(x,y)). \tag{Readings}$$

These readings specify an interpretation; they do not silently add ordinary logical axioms to the native calculus. Public $\mathsf{Q}_y B$ is interderivable with $\mathsf{D}_{v,y} B$ for an unused v. Thus Q may instead be regarded as an auxiliary closure in contexts, with the public object language consisting only of equations and D.

2.3 Every primitive inference rule

Hypothesis.

$$\frac{}{\Gamma \vdash A} \quad (A \in \Gamma). \tag{Hyp}$$

Equality transport.

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{Tr}$$

Here F is any fixed native formula template, including a compound one.

Closure elimination.

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{QE}$$

Paired elimination.

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x,y) \qquad \Gamma, \mathsf{Q}_y B(b,y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

The parameter b is fresh for Γ, A and every fixed parameter of B. The displayed local closure is discharged.

Paired case introduction.

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{DI-case}$$

Paired equation-progress introduction.

$$\frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x,y)}, \qquad E \text{ a literal numerical equation.} \tag{DI-progress}$$

In both introduction rules, z is fresh for Γ, t and every fixed matrix parameter. The displayed local assumption is discharged. A quantified formula whose innermost matrix is an equation is not a literal equation. Both case and progress introduction are available at equation roots; case introduction alone is available at compound roots.

There are no additional primitive logical axioms, equality-reflexivity axioms, numerical coverage axioms, successor-separation axioms, or induction rules in this native presentation. Structural weakening, exchange, contraction, renaming, capture-free numerical substitution, and assumption cut are admissible by induction on finite derivations. Fresh eigenparameters are renamed before a cut or substitution. The substitution preserves the root constructor, hence preserves the equation-only progress restriction.

2.4 Three elementary derived principles

Reflexivity is derived from computation. Let $r = I(0, t; u.0)$. The equation $r = t$ is I0; use it as both premises of transport with motive $v = t$ to obtain $t = t$. Ordinary explicit motives then give symmetry, transitivity, and numerical congruence.

Equation induction is derived by progress introduction with an unused witness coordinate. From $E(z) \vdash E(Sz)$ under a retained context, obtain $\mathsf{D}_{x,y}E(y)$, eliminate it to get $\mathsf{Q}_y E(y)$, supply the base $E(0)$, and specialize at the desired term. The same construction with case introduction gives numerical zero/successor cases for arbitrary native formulas. This is why case introduction cannot simply be discarded.

Let $C(n, a, b) = I(n, a; u.b)$, with u unused. Then $C(0, a, b) = a$ and $C(Sn, a, b) = b$. Under $0 = S0$, transport therefore makes every two numerical terms equal. External induction on finite formula syntax, introducing a paired formula while ignoring its local assumption, yields every native formula. This derives explosion for the designated contradiction.

3 Why the presentation is fully bi-interpretable with iPRA

The complete inherited proofs are reproduced in the appendices, with source hashes and attribution. This section states the precise dependency chain; none of the implementation changes is used as a replacement for a missing arithmetic or logical bridge.

3.1 Reconstructing the whole intuitionistic logical language

First expand every $\mathsf{Q}_y B$ as $\mathsf{D}_{w,y}B$, with w fresh and unused, using the equivalence above. On the resulting equation/D formulas, define a gate using the selector C by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_v B.$$

After fresh renaming, computation and binder congruence give $\vdash G_0 A$ and $G_{Sv}A \dashv\vdash A$. Padding by a paired binder, $A \dashv\vdash \mathsf{D}_{u,y}G_y A$, lets formulas of different depths be aligned. A finite structural recursion then constructs a selector $M_v(A, B)$ with $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$. The appendices spell out that recursion, including unequal binder depths.

Writing $p(z) = I(z, 0; u.C(u, 1, 0))$, one obtains the complete logical macros

$$\begin{aligned}
A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\
A \wedge_* B &:= \mathsf{D}_{u,z}G_z M_{p(z)}(A, B),\\
\exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_y A(x),\\
A \vee_* B &:= \exists_* x\, M_x(A, B),\\
\forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_z A(z),\\
\bot_* &:= (0 = 1).
\end{aligned}$$

All unused coordinates and binders are fresh. The fixed computations $p(1) = 1$ and $p(2) = 0$ suffice for conjunction elimination. The native rules derive every intuitionistic natural-deduction introduction and elimination rule for these macros, including the quantifier eigenvariable conditions. There is no replacement of positive disjunction or existence by a negative translation, and no excluded middle for arbitrary quantified formulas.

3.2 The ordinary realization and its arithmetic strength

Let T_I be ordinary intuitionistic first-order equality logic in the staged scalar-iterator signature, with I0, IS, contextual literal-equation induction, and the two explicit sentences

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second is the falsity bridge. It cannot be silently omitted from an ordinary theory merely because native contradiction is represented by an equation. The native rules are sound in T_I under (Readings); the derived logic, cases, induction, and explosion recover the converse interpretation.

The scalar arithmetic development proceeds in a noncircular order. First it constructs a scalar predecessor P and proves $P0 = 0$ and $P(Sn) = n$, without assuming pairing or actual-equality reflection. Then it constructs truncated subtraction and a Boolean characteristic of actual equality, proving reflection on the entire numerical domain. Truth-table constructions extend that characteristic to each fixed quantifier-free formula and recover contextual quantifier-free induction. Only then are pairing, projections, and the compiler for every fixed primitive-recursive description constructed.

For example, if π, L, H are the established pairing and projections, a primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is represented by

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))),$$
$$f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$ and the defining equations of f^*. The construction is external recursion on a fixed finite description. It postulates no uniform total evaluator for all programs.

Theorem 3.1 (The exact preserved equivalence)**.** *The native calculus of Section 2, its ordinary realization T_I, and full first-order iPRA have effective interpretation and proof translations with the following properties. Both numerical domains are the whole domain, equality is actual equality, and the numerical return isomorphisms have graph $x = y$. Both formula composites are provably equivalent to the original formulas. In particular, the theories are bi-interpretable and equiconsistent.*

Proof. Interpret scalar iteration in iPRA by ordinary primitive recursion whose update ignores the counter. Interpret every fixed primitive-recursive description in T_I by the scalar compiler. The inherited predecessor and reflection theorems supply the successor axioms and all quantifier-free induction instances, so these are interpretations of the stated theories.

Two recursions with the same initializer and extensionally equal updates are internally equal by literal-equation induction. External induction on each finite description therefore identifies its double translation with the original operation. This proves preservation of every operation by the identity graph on the full domain, not merely agreement at standard numerals. The gate/selector logical reconstruction proves the ordinary meaning of every macro. Conversely, native paired formulas and closures are interderivable with their translations in (Readings). Binder congruence and external induction on finite formula syntax give both formula composites. Contradiction is preserved by the explicit falsity bridge. Translating finite contradiction proofs in either direction gives equiconsistency. The detailed inherited arithmetic lemmas and logical derivations are in the attributed appendices. □

4 A uniform zero representation and constructed majors

The new contribution is a change of proof presentation. The numerical operations, native formulas, and primitive mathematical rules remain those specified above. In particular, this section neither removes an arithmetical axiom nor changes the domain of interpretation. It changes which finite data represent zero and how an elimination packet specifies the theorem that must be proved.

The distinction is useful here. Earlier packets used Python falsiness both as an instruction marker and as the canonical spelling of arithmetical zero. A zero initializer or predecessor consequently needed a separate proof replacing it by a nonempty term. We choose a canonical zero that is already truthy, maintain that convention during opening, and construct an equality or closure major from its component fields. The construction still checks a proof of that major.

4.1 Canonical expressions and the zero convention

Retain the inherited arity codec. Under p protected numerical slots, formula depth is $2p$, and numerical depth is $3 + 2p$. At a numerical position, coordinates $0, 1, 2$ denote zero, while $3 + 2j, 4 + 2j$ denote protected slot j, counted from the inside. Admitted negative integers are rigid parameters. The meaningful constructors and their environments are as follows:

sort	raw constructor	additional child environment
number	$[n]$	two dummy zero slots
number	$[n, a, h]$	one zero slot for n, a; one state slot for h
formula	$[s, t]$	the same numerical slots
formula	$[B]$	the two slots of D
formula	$[B, B, B]$	the one slot of Q.

Thus the common depth increment $5 - \mathrm{len}$ retains its existing meaning. Numerical binary and surplus arrays denote zero; formula surplus arrays and admitted scalar formula padding denote the fixed truth $0 = 0$. Those aliases are administrative syntax, not new arithmetical or logical symbols.

In the new canonical codec, *raw scalar* 1 *denotes numerical zero*. The number $S0$ is represented by `[1]`, and $S(S0)$ by `[[1]]`. Raw scalar 1 in a packet's mode field is instead an instruction marker; the field's role distinguishes the two uses.

Let R map an old canonical expression to the new one by replacing each numerical-zero leaf 0 by 1, leaving every constructor, protected coordinate, and rigid parameter unchanged. This operation is applied to expression fields. A raw zero used as an instruction marker remains zero.

Lemma 4.1 (Uniform truthiness)**.** *Every canonical numerical expression has a truthy raw representation. Every canonical formula is a nonempty array. The old and new representations decode to the same mathematical expression.*

Proof. Canonical zero is the nonzero scalar 1. A rigid parameter is a strictly negative integer; a protected-coordinate or substitution-hole occurrence has index at least 3. Successor and iteration are nonempty arrays. These are all numerical constructors. Equality, Q, and D are respectively binary, ternary, and unary nonempty arrays. Finally, coordinates 0 and 1 already have the same zero meaning in every numerical environment, and R changes neither a binder nor its environment. Structural induction gives the last assertion. □

Truthiness is a property of this finite syntax. A truthy term can denote zero, and a truthy equation can be false.

4.2 Opening preserves the convention

Write i_0 for the inherited opener. The new opener i_+ differs only in the branch for a scalar already admitted by the current floor and depth:

$$\texttt{a} \quad \longmapsto \quad \texttt{a or 1}.$$

The list traversal, protected-coordinate test, numerical image check, and depth increments are unchanged.

Lemma 4.2 (Canonical opening commutation)**.** *For a well-scoped canonical expression A, at a meaningful sorted root of depth d, and a canonical numerical image t,*

$$i_+(R(A), R(t), d) = R(i_0(A, t, d)). \qquad \text{(Opening commutation)}$$

The no-image version holds whenever opening does not use an image. Opening a canonical template with raw image 0 *in the new codec has the same result as opening it with raw image* 1*.*

Proof. First prove the no-image statement for closed images by structural induction. Then proceed by structural induction on the template. At a meaningful numerical position the depth is odd and at least 3. If the old leaf is zero, it is protected; the new representation and the changed preserved-scalar branch both give 1. Every other protected leaf is unchanged. An exposed leaf invokes the numerical-root check of the image, and the already proved no-image statement applies to that check.

For a successor, the same two dummy zero slots are introduced on both sides. For an iterator, the same single slot is introduced, with its inherited zero or state interpretation in the respective child. Equality changes from formula to numerical depth; the two logical binders add the same protected slots. The recursive results therefore assemble into exactly the same raw tree on both sides of (Opening commutation). A canonical formula has no scalar formula root, so scalar formula padding does not arise in this exact identity.

An image used in a numerical position is itself checked at depth 3. At every admissible floor,

$$i_+(0, \mathrm{None}, 3) = 1 = i_+(1, \mathrm{None}, 3).$$

The two raw images therefore have identical inserted results. If an image is unused, its spelling is irrelevant. □

Lemma 4.3 (Semantic opening on accepted aliases)**.** *Every successful new opening at a meaningful sorted root has the same capture-free substitution interpretation as the inherited opener.*

Proof. The only changed scalar result is a falsy value retained by the scope test, such as 0, signed floating zero, or `false`. At a meaningful numerical position that value denotes zero; so does the returned coordinate 1. At a meaningful scalar formula position both denote truth. A call below a constant-meaning surplus node concerns an ignored annotation. All other successful scalar cases and all constructor clauses are unchanged. The inherited structural semantic-opening proof consequently applies with this additional scalar case. □

The exact identity in Lemma 4.2 is deliberately restricted to canonical expressions. For arbitrary administrative arrays, a subsequent traversal can treat an ignored coordinate differently. Scalar formula padding also need not be a fixed point of raw traversal. Lemma 4.3, rather than universal raw idempotence, is the invariant used for these aliases.

Unlike reduction of $I(0, a; v.h)$ to its seed, this recoding deletes no syntax node or protected slot. In particular, it needs no de Bruijn downshift beneath nested iterators.

4.3 Constructing the elimination major

An instruction has four fields $[m, s, u, t]$. When m is truthy, construct

$$M(m,s,u,t) = \begin{cases} [s,u], & s \text{ truthy}, \\ [t,t,t], & s \text{ falsy and } t \text{ truthy}, \\ m, & s,t \text{ both falsy}. \end{cases} \qquad \text{(Constructed major)}$$

The checker proves M recursively in the retained context before using it. If opening t at u matches the current goal, its candidate replacement is the opening of t at s. The usual common continuation exports three copies of the checked major. The source-level Boolean spelling may retain the original goal when the replacement is falsy; this omission of a conversion is harmless.

Proposition 4.4 (Soundness of constructed majors)**.** *Replacing the old explicit-major pattern comparison by* (Constructed major) *preserves soundness of the elimination branch.*

Proof. If s is truthy, the checked major is exactly the equation $s = u$. A successful target comparison identifies the old goal with $F(u)$, where F is the decoded template t. The continuation's changed goal is $F(s)$, so ordinary equality transport proves $F(u)$. There is no additional assertion that some separately supplied formula has these endpoints.

If s is falsy and t truthy, the checked major is exactly $\mathsf{Q}_y F(y)$. A successfully used falsy numerical image denotes zero. Thus the proposed minor is $F(0)$, and the target is $F(u)$; this is the existing closure-elimination rule. An unsupported falsy JSON value rejects when a relevant numerical occurrence uses it. When the image is unused, zero may be chosen as its representative.

Suppose finally that s, t are falsy. The supplied major is checked, and the existing triple export gives paired elimination when it is unary. Other nonempty array majors export a surplus truth formula. There is no extra substantive target-conversion rule in this case: a successfully opened falsy formula template denotes truth, so a matching target already denotes truth. For canonical nonempty goals, the ordinary choice $t = 0$ yields scalar formula padding and cannot match the goal at all. Unsupported falsy values either reject or are irrelevant. This exhausts the branches. □

The freshness argument is unchanged: retained goals and hypotheses are checked before a new local parameter is admitted. Constructing a major does not make it a hypothesis; its recursive proof remains necessary. Local allocation versus a global allocation counter is a separate implementation choice, justified by the inherited scope argument.

4.4 Completeness without zero repairs

Theorem 4.5 (Direct canonical compilation)**.** *Every native derivation using the unrestricted I0 and IS computation schemas has a canonical certificate under the new representation. No computation node requires a proved nonempty replacement for zero. On decoded terms and formulas, this compilation is the identity.*

Proof. Use R on expression fields and preserve instruction markers. The following packets implement

the primitive inferences; fields without a mathematical role are fixed as displayed:

inference	(m, s, u, t)
transport through F	$(1, R(s), R(u), R(F))$
Q-elimination from $\mathsf{Q}B$	$(1, 0, R(u), R(B))$
D-elimination	$(R(\mathsf{D}B), 0, 0, 0)$
I0 with seed a and update h	$(0, 0, R(a), R(h))$
IS at predecessor n	$(0, R(n), R(a), R(h))$
DI-case with witness w	$([], 0, 0, R(w))$
DI-progress with witness w	$(0, 0, 0, R(w))$.

The paired matrix and its variable environments are those specified by the goal, as in the inherited certificate compiler.

By Lemma 4.1, an equality source $R(s)$ is always truthy, including when $s = 0$. A canonical closure matrix is nonempty. The constructed majors therefore select exactly the required equality and closure theorems. Their proof premises are compiled recursively; Lemma 4.2 gives the required minor conclusions. For paired elimination, the checked unary major exports exactly its witness closure.

The initializer $R(a)$ is always truthy, so every computation is distinguished from paired introduction. In I0, the generated raw zero counter is converted to the canonical scalar 1 when the supplied equation is opened. In IS, even predecessor zero has truthy representation 1, so the generated counter is `[1]`, precisely the canonical successor of zero. The update application commutes with R by Lemma 4.2. These are direct instances of the two computation schemas.

The two introduction modes retain distinct falsy markers. Their zero-instance images become canonical zero automatically; fresh negative parameter images remain unchanged. Recoding leaves the matrix's root constructor unchanged, so progress remains restricted to literal equations. Hypothesis closure follows from the same opening-commutation lemma. Induction on the native derivation now supplies the complete certificate. Extra surplus truth entries exported by equality or closure majors need not be used by that certificate.

For noncanonical computation packets, the inherited known-zero-instance argument still supplies soundness: specialize all newly admitted parameters to numerical zero and cut in the resulting computation theorem. Lemma 4.3 preserves that argument. It does not claim that a generated hypothesis is uniformly true at a new arbitrary parameter.

Finally, Lemma 4.1 says that decoding every translated expression returns the original expression. The compilation therefore changes neither its numerical interpretation nor its formula interpretation. □

Corollary 4.6 (Transfer of the full interpretation)**.** *The canonical consequence relation checked by the new presentation is the inherited native consequence relation. Its whole-domain bi-interpretation and equiconsistency with* iPRA *transfer unchanged, including the formula return interpretations.*

Proof. Theorem 4.5 proves completeness on the canonical image. Lemma 4.3, Proposition 4.4, and the inherited arithmetic, introduction, and scope arguments prove soundness. The decoding map is the identity on the native expressions, so compose it with the existing two interpretations. Their internal proofs of both composites remain the same. □

4.5 Exact certificate-size benefits

Let $|X|$ denote the UTF-8 byte length of compact JSON with no whitespace, and use one-character scalar flags consistently in the compared packets. Then

$$|[X, Y]| = |X| + |Y| + 3, \qquad |[B, B, B]| = 3|B| + 4.$$

Replacing an equality major field $[s, u]$ by flag 1, while retaining the endpoint fields already present in the packet, saves exactly

$$\boxed{|s| + |u| + 2} \qquad \text{(Equality saving)}$$

bytes. Replacing a closure major field $[B, B, B]$ by flag 1, while retaining its matrix field, saves exactly

$$\boxed{3|B| + 3} \qquad \text{(Closure saving)}$$

bytes. No inference node or proof premise is removed by these two field changes. The paired-major field is unchanged.

For the same native proof tree with the same parameter allocation, zero recoding changes a one-character digit to another one-character digit. Thus the two displayed savings add exactly over its equality and closure elimination packets. The outer certificate-array punctuation is unchanged because the packet count is unchanged. This exact comparison can, for example, be applied to a common proof already satisfying the old initializer restriction. It also applies to the fully repaired old proof before removing its repairs.

There is a separate proof-node benefit. In the certified-zero construction let

$$C = I(S(S0), S0; v.0).$$

The supplied repair replaces an excluded computation leaf by a transport with the two computation premises $C = 0$ and $E(C)$. It therefore replaces one node by three. If an unrestricted derivation has N non-hypothesis nodes and r such computation leaves, the displayed tree transformation has

$$N_{\text{old repair}} = N + 2r, \qquad N_{\text{new}} = N, \qquad N + 2r \le 3N. \qquad \text{(Tree counts)}$$

The new construction directly serializes the original computation nodes. Where neither compiler closes an additional goal by a previously retained hypothesis, the same equations count instruction packets. Context reuse can remove packets from either serialization, so (Tree counts) is not a theorem about shortest accepted certificates.

If one shares a single $C = 0$ node in a DAG and $r > 0$, the old displayed repair instead has $N + r + 1$ distinct nodes. The new proof still uses N. This is again a comparison of the specified constructions, not a lower bound on all possible old derivations.

Both transformations have linear node overhead. Constructing majors copies already present fields at most three times, and C is a fixed finite term. Their syntactic work is consequently linear in the fully expanded encoded proof data, apart from the ordinary integer-name bookkeeping. This statement does not remove the inherited triplicate-closure codec's potential exponential expansion relative to a different, shared representation of formulas. When removal of repairs changes numerical parameter labels, exact whole-file byte comparisons must also account for that renaming.

A complete small example. For $I(0, 0; v.0) = 0$, the following compact old certificate has three instruction packets and 97 bytes. It uses the supplied three-node repair and a scalar-zero computation marker accepted by the old checker:

```
[[0,0,[[[0]],[0],0],0],[0,[0],[0],0],
 [[[[[0]],[0],0],0],[[[0]],[0],0],0,[[0,5,0],3]],
 [[0,0,0],0]]
```

The line breaks above are for display and are excluded from the measurement. The new certificate is

```
[[0,0,1,1],[[1,1,1],1]]
```

with one instruction packet and 23 bytes. Both complete programs accepted their respective certificate. The mathematical conclusion is identical under the two decoders. The 74-byte reduction is exact for this pair; it does not assert that 97 bytes is the minimum old certificate.

4.6 What the result establishes

The corpus already recognizes nonempty zero terms and scalar zero aliases. The new combination is the uniform canonical choice of a truthy scalar zero, its preservation when the checker creates base instances, and its use in constructing elimination majors. The proved benefits are direct zero computation and removal of redundant major fields. These are changes to proof presentation.

The native axiomatic basis is unchanged. Kernel-size measurements must therefore distinguish this contribution from the inherited scope discipline, Boolean source factoring, and other implementation choices. Neither the representation theorem nor the certificate counts establish an absolute simplest axiomatization or a major new foundation of arithmetic.

5 The complete checker and its trust boundary

The following is the complete constructed-major byte profile. The uniform zero-only profile retains the old explicit-major comparison instead. The readable version expands the same stages and is measured separately.

```
import json

def c(a, g=[], k=0):
    i = lambda a, t=0j, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a or 1 if k <=
    a < d else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c((m := ([s, u] if s else t and [t] * 3 or m)), g, k)
        a = i(t, u) == a and i(t, s) or a
    h = m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == m) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

Input is one JSON array. Its last item is the requested formula; earlier items are instruction packets consumed from the end. Hypothesis closure uses no packet. Unused prefix instructions are permitted. Acceptance requires normal exit and exactly the output `ACCEPT`. An exception rejects. The programs contain no `assert` whose checking disappears under Python's optimization option.

5.1 The inherited arity decoder

Formula arrays of length one, two, and three represent respectively a paired formula, an equation, and an auxiliary closure. Canonical closures are triplicated: $[B, B, B]$. Empty and surplus formula arrays denote the fixed truth equation. Numerical arrays of length one and three represent successor and scalar iteration; other numerical arrays denote zero. Scalar formula padding denotes truth. These are serialization meanings, not extra independent predicate symbols.

At a numerical root the depth is three. Coordinates $0, 1, 2$ all denote zero; under protected slots, coordinates $3 + 2j$ and $4 + 2j$ denote slot j counted from the innermost slot. Negative allocated coordinates are rigid parameters. The traversal adds $5 - \mathrm{len}(a)$ at every array. This gives four for a unary paired binder or a successor with dummy slots, three for the sort change at equality, and two for a closure or iterator. The full inherited sorted decoder and successful-opening argument are included as `reference/inherited_arity_checker.tex`.

The new scalar clause sends preserved zero to raw 1. This value lies inside the same numerical-zero interval at every meaningful position. It does not move a binder, change a de Bruijn coordinate, or remove a dummy slot. It has a true Python Boolean value; the mathematical successor of zero is the unary array [1], not raw scalar 1.

5.2 Lexical scopes and shared continuation selection

Each invocation of the local checker receives its current floor k. The opener closes over that local variable. The major is checked at the old floor. A child major proof may allocate its own local parameters, but does not change the parent's floor. Then the parent decrements its floor and checks its continuation at the new floor. The previously checked goal and retained context cannot mention that new rigid name. Reuse of names in disjoint completed scopes is ordinary alpha-renaming; a single global fresh-name counter is unnecessary. This technique is inherited from the local-scope draft.[4]

The context default is an empty list, but no checker mutates it: every continuation passes a newly concatenated list. Thus the usual mutable-default pitfall does not occur. The source uses `0j` as its missing-image sentinel. JSON cannot supply that value; if a meaningful unbound numerical hole requests it, the numerical comparison rejects it.

The Boolean expression constructing the next hypothesis has three ordered alternatives. A nonempty proved major supplies its tripled export. Otherwise a nonempty initializer supplies a computation equation, which is always a nonempty pair. Otherwise exact unary introduction supplies its opened matrix. Canonical arithmetic markers are raw 0, not JSON null. This change excludes some old raw spellings while preserving the entire canonical consequence relation. The paired introduction call still enforces the unary shape through argument binding; there is no unchecked introduction of an arbitrary equality or surplus connective.

5.3 Fresh computation assumptions are not universal axioms

The generated arithmetic hypothesis can contain the newly admitted name b. In general, only its zero instance is a theorem. For a retained context and conclusion independent of b, the rule

$$\vdash H(0), \qquad \Gamma, H(b) \vdash A \quad \Longrightarrow \quad \Gamma \vdash A$$

is admissible: substitute zero for b in the continuation and cut in $H(0)$. For the permissive floating-point input aliases, substitute zero also for every additional fresh name in the newly admitted unit

interval that occurs in the finite derivation. They are not all identified with the designated integer eigenparameter.

The constructed source and recursive-image check identify that zero instance with I0 or IS. The new zero normalization preserves its decoded meaning. Claiming that the generated $H(b)$ is uniformly true would be incorrect; neither the implementation nor the proof requires that claim.

5.4 What is and is not certified

The soundness argument is a mathematical induction on finite accepting checking trees, using the sorted-opening, scope, computation-supply, and native-rule lemmas. Canonical completeness is an effective compilation of finite native proofs. The included execution tests inspect substantial instances and failure modes; they are not a Lean or Rocq proof of those universal statements. All actual Python programs also have finite memory and stack limits and can reject sufficiently large otherwise valid proofs.

6 Reproduction and validation

All 504 expected acceptance/rejection outcomes passed across seven complete programs in ordinary and optimized Python. The suite contains 36 cases per program per mode. It includes direct zero computation, zero witnesses, zero-source equality transport, nested closures, an actual existential witness, equation induction under a retained hypothesis, and transport under a quantified formula. Negative controls cover false conclusions, wrong computation endpoints, future-parameter capture, invalid witnesses and specialization targets, forbidden compound progress, missing fields, and malformed input.

Four deliberate guard-mutation checks also behaved as expected. Removing major verification admits a forged proof of $0 = 1$; removing the literal-equation restriction admits the forged compound-progress certificate. Each mutant was checked in both interpreter modes. These checks show that the suite exercises the guards; they are not a replacement for their mathematical soundness proof.

Across the sixteen successful non-neutral named comparison examples, the measured certificates have the following sizes:

Certificate representation	Commands	Compact JSON bytes
Matched historical	120	5,029
Uniform zero	90	3,801
Uniform zero and constructed majors	90	2,955

This sample has 25% fewer instruction packets and approximately 41.24% fewer serialized bytes in the constructed-major format. These are selected regression examples, not a representative workload or a universal compression ratio. The contextual-induction example changes from 48 commands and 2,359 bytes to 40 commands and 1,437 bytes. Both direct zero computations change from three commands to one.

The complete per-case data are in `evidence/validation.json`, and the matching certificates are under `examples/old`, `examples/zero`, and `examples/constructed`. All measured local-scope certificates were emitted by the corresponding local-scope producer. No claim of raw-file compatibility with a globally allocated parameter convention is needed.

The named proof producer is outside the trusted checker and never imported by it. It describes symbolic rule premises, records hypotheses and scopes, and serializes certificates. The old producer retains the published certified-zero repair; the new producers serialize mathematical zero directly and perform no such expansion. Baseline and new certificates are checked by their respective complete programs, with both ordinary and optimized Python.

Source measurement parses the entire delivered Python module, removes genuine docstrings, normalizes it with `ast.unparse`, and includes one trailing newline. Significant tokens omit layout, comments, encoding, and end markers. AST counts include the entire module. Maximum-compression gzip uses compression level nine, timestamp zero, and no original filename. Both raw-source and normalized-source gzip sizes, source hashes, and runtime versions are recorded. Gzip is informational, not an objective of the search.

```
python3 verify.py
python3 measure.py
python3 build.py
```

The first command checks the supplied positive and negative certificates. The second reproduces the size table. The third compiles this document twice with `pdflatex`. Neither compilation nor checking requires network access. A TeX installation with the listed standard packages is required to rebuild the PDF.

7 Further research questions

1. **Can constructed premises pay for themselves in the kernel?** The present fully matched construction adds six source bytes after uniform zero. An improvement should remove that cost without introducing an unaccounted witness condition, hidden interpreter, or altered induction rule. The exact source ablation in this report provides a fixed target.
2. **Can one remove a mathematical rule rather than re-encode it?** The present two computation schemas and case/progress distinction remain. A smaller foundation must recover every lost rule and both formula composites. The existing countermodels to deleting atomic case introduction and to unsafe paired-witness specialization remain relevant.
3. **Can zero computation be made definitional without paying more for substitution?** The rewrite $I(0, a; u.h) \to a$ has an elementary normal-form presentation. In the current arity codec, returning the raw seed without removing its dummy slot is incorrect. A different binder layout might make this conversion cheaper; all added scope machinery must be counted. The broad conversion approach already occurs in the compact-kernel draft.
4. **Can the successful-trace soundness theorem be formalized compactly?** A useful formal target is a sorted raw-tree decoder, the opening lemma, the fresh-zero-instance supply lemma, and the canonical compiler theorem. This would strengthen assurance much more than another one-byte rearrangement of the Python source.
5. **What should the optimization objective include?** The current minimum is an empirical source-size frontier. A reproducible benchmark should keep interface, language, malformed-input conventions, scope model, and charged dependencies fixed, and report certificate size and checking work alongside checker size. A two-byte literal-file improvement can vanish under a matched implementation comparison.

Attribution and the full inherited bridge

The following five sections are reproduced unchanged from the `ipra_certified_zero_2026-11-10.zip` archive at the pinned commit. They are inherited mathematics, not new results of this report. Their original member paths and SHA-256 hashes are recorded in `evidence/inherited_provenance.json`; the repository license is included as `reference/ProveIt_LICENSE.txt`. They provide the complete logical reconstruction, ordinary realization, scalar predecessor, actual-equality reflection, and primitive-recursion compiler on which the present exact iPRA equivalence depends.[2]

The first appendix uses an older clause notation. The clause $\langle B(0); y.B(y)\rangle$ is the public closure $\mathsf{Q}_y B$. An ordinary assumption F corresponds to the constant clause $\langle H; v.F\rangle$ with H a fixed computation theorem; it does not mean the tautological closure $\mathsf{Q}_v F$. The public-Q interface and clause interface are equivalent by the derived unused-witness construction. The other appendices are used with that explicit identification.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y} A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [9] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n,a;u.h), \qquad I(0,a;u.h) = a, \qquad I(Sn,a;u.h) = h[I(n,a;u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n,a,b) = I(n,a;u.b), \qquad Z(n) = C(n,1,0), \qquad a+b = I(b,a;u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4,$$
$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [10]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion.

In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$
$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$
$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$

$$(Sx) + y = S(x + y), \tag{32}$$

$$d(Sx, Sy) = d(x, y), \tag{33}$$

$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b+1 = S(b+0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$
$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [11]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt current draft collection*, commit `76a2651601d17b34d46753f189ea026a21ce8d29`. https://github.com/VladimirReshetnikov/ProveIt/tree/76a2651601d17b34d46753f189ea026a21ce8d29/docs/drafts.

[2] *Seed-selected computation certificates for iPRA*, `ipra_certified_zero_2026-11-10.zip`, in [1]. Source of the published byte minimum, simultaneous nonempty-zero repair, comparison producer, and attributed bridge appendices.

[3] *An arity calculus for iPRA*, `ipra_arity_calculus_2026-11-10.zip`, in [1]. Unary paired formulas, triplicated closures, and the $5 - \text{arity}$ opening convention, reproduced in the certified-zero archive.

[4] *Local scopes for iPRA*, `ipra_local_scopes_2026-11-10.zip`, in [1]. Lexical eigenparameter allocation and certificate renaming.

[5] *A smaller complete iPRA checker and an obstruction to positive axiomatizations*, `ipra_compact_kernel_2026-11-10.zip`, in [1]. Includes the previously investigated definitional conversion approach and its scope-accounting obligations.

[6] Inherited generated-assumption and known-instance supply interface, reproduced and attributed in [2, 3].

[7] *Constructing computation sources for iPRA*, `ipra_constructed_computation_2026-11-10.zip`, in [1].

[8] The complete pinned current and embedded draft corpus, as inventoried in [1] and `evidence/corpus_metrics.json`.

[9] Inherited scalar-predecessor development, reproduced as `appendices/scalar_predecessor.tex`; see [2].

[10] Inherited equality-reflection development, reproduced as `appendices/equality_reflection.tex`; see [2].

[11] Inherited pairing and primitive-recursion development, reproduced as `appendices/primitive_recursion.tex`; see [2].

[12] Jeremy Avigad. *Proof mining*, lecture slides, 2004, slide 13. https://www.andrew.cmu.edu/user/avigad/Talks/asl04.pdf. Used for the background distinction between first-order classical, first-order intuitionistic, and quantifier-free PRA presentations.